UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

INFORMATICS INSTITUTE OF TECHNOLOGY

Client-Server Architectures

5COSC022C.2

Conceptual Report

- Name : W.G. Thewan Sewmin
- UOW ID : w2120081
- Student ID : 20240693

Table of Contents

Part 1: Service Architecture & Discovery 3
1.1 JAX-RS Resource Lifecycle 3
1.2 Hypermedia and HATEOAS 3
Part 2: Room Management 4
2.1 Returning IDs vs. Full Objects 4
2.2 Idempotency in DELETE Operations 4
Part 3: Sensor Operations 5
3.1 Technical Consequences of Media Mismatches 5
3.2 Query Parameters vs. Path Parameters 5
Part 4: Sub-Resource Locators 5
4.1 Architectural Benefits of Sub-Resource Locators **5**
Part 5: Error Handling and Logging 6
5.2 Semantic Accuracy: 422 vs. 404 6
5.3 Cybersecurity Risks of Stack Traces 6
5.4 Advantages of JAX-RS Filters 6

Part 1: Service Architecture & Discovery

1.1 JAX-RS Resource Lifecycle

As default, the resource classes in JAX-RS follow a Request Scoped Lifecycle. The design guarantees that the container creates an instance of the resource class per each HTTP request received. Once the response is sent, the resource class instance is garbage collected. They are single use and non thread safe, that is why instances of the resource cannot have any information stored in themselves.

To overcome this issue, the application has a dedicated MockDataStore which holds all the data needed. For this particular implementation, use ConcurrentHashMap from the Apache Tomcat library. It is used since the application can have many concurrent users accessing the API concurrently. This fact is crucial in order to provide data consistency by preventing data from getting into the state of "race conditions".

1.2 Hypermedia and HATEOAS

HATEOAS links play a vital role in RESTful API design due to their ability to enable the self description of an API. Instead of having to study voluminous documentation, developers only need to inspect the links within the response (_links provided in the JSON response) to understand how to proceed further. The advantage of this approach over static documentation lies in its scalability; the API can evolve without the developer needing to modify their code as long as the links can guide them through any changes to URLs.

Part 2: Room Management

2.1 Returning IDs vs. Full Objects

In deciding the type of response payload that will be used in transmitting the room resources, two options were available, ID or object transmission. While object transmission may lead to increased traffic and bandwidth usage, While object transmission reduces the number of round trips needed to retrieve data, ID transmission is sometimes considered to save bandwidth.

In designing the Smart Campus API, Full Object transmission was considered in order to provide users with an improved experience in terms of low latency. In doing so, the app ensures that fewer calls to the server are required for a user to receive all the details about the room resources in one request.

2.2 Idempotency in DELETE Operations

In this case, the DELETE operation is fully idempotent as well. During its first call, when the deletion succeeds, the server responds with HTTP 204 No Content code. If a client performs the same action once again, then the server returns HTTP 404 Not Found response.

Although HTTP statuses are different now, the overall state of the server remains unaffected by this modification since it deletes resources anyway. It guarantees the proper and safe functioning of the API as the user will be able to perform this request multiple times without experiencing any adverse effects.

Part 3: Sensor Operations

3.1 Technical Consequences of Media Mismatches

"@Consumes(MediaType.APPLICATION_JSON)" specifies a stringent requirement on the nature of the data transfer protocol used by the consumer. It guarantees that the API will only attempt to interpret the received message payload in a structured JSON format. Any attempt by the client to provide data in formats that are not compatible with the JAX-RS framework such as text/plain and application/xml is immediately detected due to incompatible headers.

The JAX-RS runtime will automatically return an HTTP 415 Unsupported Media Type status code, which prevents the processing of the data by the back end logic due to the inability to interpret data of a 'malformed' nature."

3.2 Query Parameters vs. Path Parameters

The architectural approach makes a difference between path parameters (which identify resources, /rooms/ROOM-101) and query parameters (which filter the collection, for example: ?type=Temperature).

It is widely accepted that using query parameters for filtering is much better as it provides a flexible and non hierarchical approach to searching. This avoids having a deeply nested URL structure and makes it easier to apply several filters without creating custom path mapping rules for every type of filtering task.

Part 4: Sub Resource Locators

4.1 Architectural Benefits of Sub-Resource Locators

This application employs the Sub Resource Locator architectural pattern to keep the code base clear and manageable. Through this approach, the separation of concerns is ensured since the complex process of sensing is handled through the use of a separate resource known as the SensorReadingResource.

The code becomes much more maintainable, scalable, and testable when God Classes classes that contain numerous unassociated functions are avoided. The use of this approach is comparable to the natural hierarchy in the physical world in relation to the campus, where sensors and the related data points are distinct yet linked entities.

Part 5: Error Handling and Logging

5.2 Semantic Accuracy: 422 vs. 404

For API testing to succeed, error messages should be semantically accurate. When an error message reads “404 Not Found,” it indicates that there is something wrong with the URL path. Conversely, when an HTTP status code reads “422 Unprocessable Entity,” it means that even though the request is acceptable, it does not make logical sense.

If, for example, a user attempts to attach a device to a non existent Room ID, he or she will receive a 422 error response. In such a case, the error response tells the developer that the endpoint is valid, but the reference information within the JSON body is invalid.

5.3 Cybersecurity Risks of Stack Traces

It's very dangerous to let the public see internal Java stack traces. These traces are like a "blueprint" of the server because they show private information like internal file paths, library versions (like Jersey 2.32), and specific class names.

Attackers can use this information to find known exploits or weaknesses in those specific versions of software. The GlobalExceptionMapper in the Smart Campus API lowers this risk. This "Safety Net" catches all unexpected errors and replaces them with a clean, professional JSON error message, keeping the server's inner workings hidden.

5.4 Advantages of JAX-RS Filters

This project implements a LoggingFilter that is injected with the help of @Provider to solve issues that impact the entire system architecture. Such an approach complies with one of the basic principles of software development, namely the Don't Repeat Yourself (DRY) principle.

In this case, the filtering process helps avoid manual copying and pasting of the logging operations within each method inside the resources. It will observe each request and response automatically and send them into the Tomcat Catalina logs. This helps ensure that the system will always be observable and minimize chances for human error while maintaining the core business logic intact.